

Hands On 1: HTML and GitHub

The objective of this activity is for students to develop practical skills in creating HTML documents and deploying the finished document group through GitHub for accessibility. This activity is useful in strengthening both technical usage and content organizing skills while providing real-world experience in developing and publishing static web content.

Instructions:

Your main task for this activity is to (1) create at least 2 HTML files connected through links and (2) deploy it in GitHub Pages to be accessible in the cloud. Your HTML files should contain actual (no dummy) details about you which can serve as your personal page. **NOTE:** *These details should only include those that you are comfortable sharing in public.* Use your creativity on what contents should be displayed. As for the technical requirements, your HTML files should use:

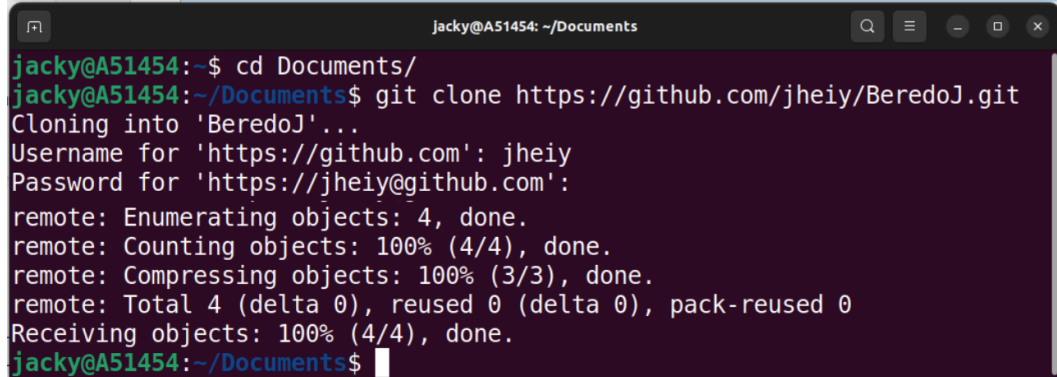
- semantic elements;
- headers;
- a table;
- a form with different input types;
- links - (1) within the page, (2) to another page you created, and (3) external site; and
- media - (1) image, (2) audio, and (3) video

NOTE: Strictly no CSS or any styling for now. Just use plain HTML. No similar webpage / contents should exist in class. The more unique your page is, the better. Once done with the implementation, deploy your static website using GitHub Pages.

Using Git and GitHub:

1. *[No need to do this in the laboratory computers, Git should already be installed. Use this for your personal machines.]*
Install git on your local machine by following the detailed tutorial depending on your operating system in this link -> <https://www.linode.com/docs/guides/how-to-install-git-on-linux-mac-and-windows/>
2. Create an account on github.com using your preferred email.
3. In your command prompt (CMD), git may ask you to set your GitHub username and email. To do this, follow the git commands below:
 - a. `git config --global user.name "<username>"`
 - b. `git config --global user.email "<email>"`
4. To clone a repository to your machine using the command line (*if you have no repository yet, refer to the Step 1 of "To deploy" instructions*):

- a. Open your newly created repository, click the green button '**Code**' and choose '**HTTPS**' from the mini tab, then click the copy button beside the link.
- b. On your command prompt, change the directory (folder) to the location where you want the cloned directory to be saved using the `cd <folder _name>` command.



```

jacky@A51454: ~/$ cd Documents/
jacky@A51454:~/Documents$ git clone https://github.com/jheiy/BeredoJ.git
Cloning into 'BeredoJ'...
Username for 'https://github.com': jheiy
Password for 'https://jheiy@github.com':
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 4 (delta 0), reused 0 (delta 0), pack-reused 0
Receiving objects: 100% (4/4), done.
jacky@A51454:~/Documents$

```

- c. Type **git clone** in the command prompt, paste the link you copied then press 'Enter', example:
 - `git clone https://github.com/jberedo/AY2627T1-CCAPDEV-HO1-BeredoJL.git`
 - d. Your GitHub username and password will be asked. Username is the one you set from Step 3. For the password, you have to create a token first in the GitHub webpage:
 - Click your profile picture located on the top-right then choose **Settings**.
 - Go to **Developer settings** at the bottom of the list on the left then to **Personal access tokens -> Tokens (classic)**
 - Click '**Generate new token (classic)**' just below your profile picture
 - In the 'Note' field, just type in anything eg. **CCAPDEV**
 - In Expiration, choose 'Custom' then set until **Dec 31**.
 - Tick all the checkboxes.
 - Then 'Generate token'.
 - Make sure to **copy and save** to a file the generated token displayed with a light green background as it will not be visible after. You will use this every time you update the repository in place of a password.
5. In your computer, check that a folder of your repository name is created. If there are files already existing in the repository you cloned, it should be found inside your folder.
 6. You may now make changes to the repository (folder) such as editing the existing files, deleting files, or adding a new one.
 7. It is recommended to push changes every time you complete an important task.
 8. Here are some useful git commands:
 - a. [to see changes/updates in your local repository] **git status**
 - b. [to add an updated file to the GitHub repository] **git add <filename>**
** You may type git status again to see that there are 'Changes' now to be committed to the GitHub repository*
 - c. [to add ALL changes to the GitHub repository] **git add ***
 - d. [to commit changes to GitHub repository after add command] **git commit -m "<commit message>"**

* Commit message should be a short description of your update, for example "Added navigation"

- e. [to push changes to GitHub repository] **git push**

* DO NOT forget to push your changes after every commit to sync your local and GitHub repository copies

* Username and password (token) will be asked in this command again

* You may now refresh your GitHub repository to see the changes.

To deploy:

1. Go to GitHub and create a new repository following the naming convention (wrong name won't be checked): **AY2627T1-CCDEVAP-HO1-SurnameInitials**

Make sure the visibility is 'Public' otherwise I won't be able to access and check it.

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

1

General

Owner *

 jberedo

Repository name *

AY2627T1-CCAPDEV-HO1-BeredoJL

AY2627T1-CCAPDEV-HO1-BeredoJL is available.

Great repository names are short and memorable. How about [friendly-octo-computing-machine?](#)

Description

0 / 350 characters

2

Configuration

Choose visibility *

Choose who can see and commit to this repository

Public

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

Off

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

No .gitignore

Add license

Licenses explain how others can use your code. [About licenses](#)

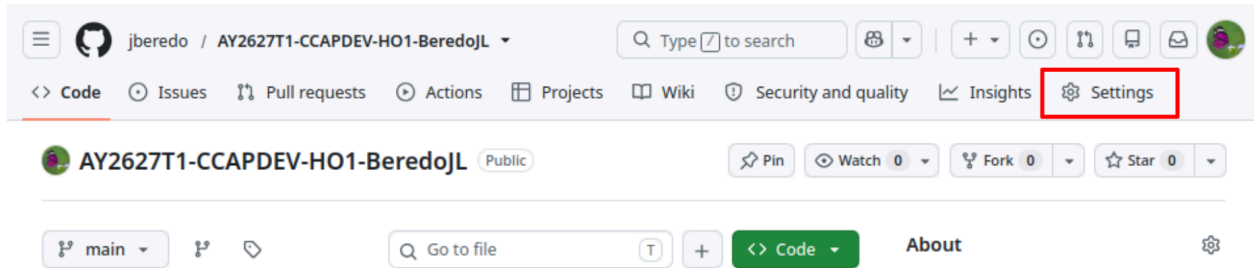
No license

Create repository

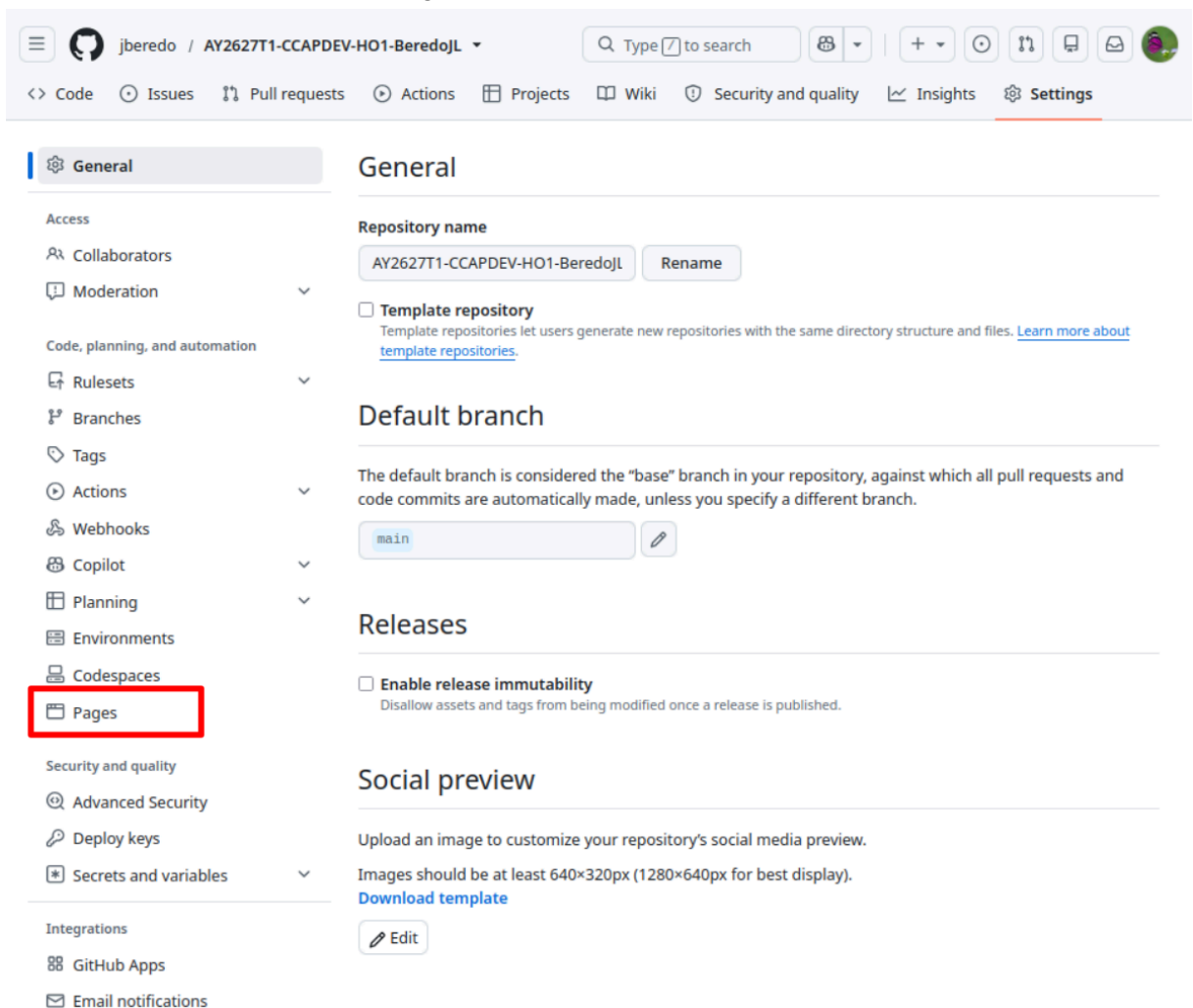
2. Upload your HTML files to the repository. This assumes you already installed and is familiar with Git and how to manage files in it otherwise, refer to this tutorial:

<https://tinyurl.com/basic-git-tutorial>

3. Next, click the **'Settings'** at the top of the repository.



4. Select **'Pages'** from the left-hand menu. It's located in the **'Code, planning, and automation'** section of the settings.



5. Use the combo box under **'Build and deployment'** and select **'Deploy from a branch'**. Under **'Branch'**, use the combo box to select **main** as the branch to deploy from. Then click **'Save'**. This publishes the website from the `main` branch and makes it publicly available.

GitHub Pages

[GitHub Pages](#) is designed to host your personal, organization, or project pages from a GitHub repository.

Build and deployment

Source

Deploy from a branch ▾

Branch

GitHub Pages is currently disabled. Select a source below to enable GitHub Pages for this repository. [Learn more about configuring the publishing source for your site.](#)

main ▾

/ (root) ▾

Save

- Now go back to the repository's home page and click the **'github-pages'** under **'Deployments'**. Then check your deployed page by clicking the provided link.

The screenshot shows the GitHub repository interface for user jberedo and repository AY2526T3-CCDEVAP-HO1-Beredo. The 'Deployments' tab is active, displaying a list of deployments. Under the 'Environments' section, 'github-pages' is selected. The 'Latest deployments' section shows a deployment named 'github-pages' that was last deployed 2 minutes ago. The deployment link is <https://jberedo.github.io/AY2526T3-CCDEVAP-HO1-Beredo/>.

- Note that any changes to the branch will be reflected on the deployed link.